

DAM2 UE04 Pokemon

Buchinger, Braun

Setup

```
In [61]: import re
import warnings
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.preprocessing import StandardScaler
from sklearn.metrics import pairwise_distances, silhouette_score
from sklearn.cluster import AgglomerativeClustering
from sklearn.manifold import TSNE
from sklearn.ensemble import IsolationForest
from sklearn.neighbors import LocalOutlierFactor

warnings.filterwarnings("ignore")

OUT = Path("figures")
OUT.mkdir(exist_ok=True)
```

1. Daten laden

`height` und `weight` sind als Strings gespeichert ("0.7 m" , "6.9 kg"), daher parsen wir die führende Zahl in numerische Spalten. Alles andere ist bereits numerisch oder binär (0/1) vorhanden.

```
In [62]: def parse_number(s):
m = re.search(r"([\d.]+)", str(s))
```

```
        return float(m.group(1)) if m else np.nan

df = pd.read_csv("pokedex.csv")
df["height_m"] = df["height"].apply(parse_number)
df["weight_kg"] = df["weight"].apply(parse_number)

STAT_COLS = ["HP", "attack", "defense", "sp. attack", "sp. defense", "speed"]
SIZE_COLS = ["height_m", "weight_kg"]
TYPE_COLS = [c for c in df.columns if c.startswith("type_")]

print(f"{len(df)} Pokemon, {df.shape[1]} Spalten")
print(f"Type-Spalten ({len(TYPE_COLS)}): {[t.replace('type_', '') for t in TYPE_COLS]}")
df.head()
```

898 Pokemon, 33 Spalten

Type-Spalten (17): ['fire', 'water', 'electric', 'grass', 'ice', 'fighting', 'poison', 'ground', 'flying', 'psychic', 'bug', 'rock', 'ghost', 'dragon', 'dark', 'steel', 'fairy']

Out[62]:

	id	name	HP	attack	defense	sp. attack	sp. defense	speed	species	description	...	type_psychic	type_bug	type_rock	type_ghos
0	1	Bulbasaur	45	49	49	65	65	45	Seed Pokémon	Bulbasaur can be seen napping in bright sunlig...	...	0	0	0	(
1	2	Ivysaur	60	62	63	80	80	60	Seed Pokémon	There is a bud on this Pokémon's back. To supp...	...	0	0	0	(
2	3	Venusaur	80	82	83	100	100	80	Seed Pokémon	There is a large flower on Venusaur's back. Th...	...	0	0	0	(
3	4	Charmander	39	52	43	60	50	65	Lizard Pokémon	The flame that burns at the tip of its tail is...	...	0	0	0	(
4	5	Charmeleon	58	64	58	80	65	80	Flame Pokémon	Charmeleon mercilessly destroys its foes using...	...	0	0	0	(

5 rows × 33 columns



Aufgabe 1 - Distanzfunktion

Ein Pokemon wird durch drei sehr unterschiedliche Arten von Attributen beschrieben, weshalb eine einzelne Metrik auf dem rohen Feature-Vektor wenig sinnvoll ist. Stattdessen bauen wir **drei normalisierte Distanzblöcke** und kombinieren sie mit Gewichten.

Block	Features	Metrik	Begründung
Stats	6 Basis-Stats (HP, Atk, Def, SpA, SpD, Spe)	standardisierte Euklidische	kontinuierlich, nach Z-Standardisierung vergleichbare Größenordnungen
Types	17 One-Hot Type-Spalten	Jaccard	Mengenähnlichkeit; "Water+Ice" sollte nahe an "Water+Flying" liegen, aber weit weg von "Fire+Rock" (berücksichtigt Schnittmenge)
Size	Größe, Gewicht	standardisierte Euklidische auf $\log_{10}$ -Werten	starke Tails; Log dämpft Pokemon wie Wailord, die sonst die Metrik dominieren

Jeder Block wird auf etwa $[0, 1]$ skaliert und dann kombiniert:

$$D = w_{\text{stats}} D_{\text{stats}} + w_{\text{types}} D_{\text{types}} + w_{\text{size}} D_{\text{size}}$$

Standardgewichte $(0.5, 0.4, 0.1)$ - die Stats bekommen das größte Gewicht (informationsreichstes Signal), Types ein starkes Sekundär-Votum, Größe einen kleinen Zusatz-Impuls.

```
In [63]: def build_distance_matrix(df, w_stats=0.5, w_types=0.4, w_size=0.1):
# Stats - standardisierte Euklidische
stats_z = StandardScaler().fit_transform(df[STAT_COLS].to_numpy(float))
d_stats = pairwise_distances(stats_z, metric="euclidean")
d_stats /= d_stats.max() or 1.0

# Types - Jaccard auf One-Hot-Zeilen
d_types = pairwise_distances(df[TYPE_COLS].to_numpy(int), metric="jaccard")

# Size - Log-skaliert, dann standardisierte Euklidische
size_z = StandardScaler().fit_transform(np.log1p(df[SIZE_COLS].to_numpy(float)))
d_size = pairwise_distances(size_z, metric="euclidean")
d_size /= d_size.max() or 1.0

D = w_stats * d_stats + w_types * d_types + w_size * d_size
np.fill_diagonal(D, 0.0)
return (D + D.T) / 2 # Symmetrie gegen Fließkomma-Rauschen erzwingen

D = build_distance_matrix(df)
print(f"Distanzmatrix: Form={D.shape}")
print(f"min={D.min():.3f}, mean={D.mean():.3f}, max={D.max():.3f}")
```

Distanzmatrix: Form=(898, 898)
min=0.000, mean=0.516, max=0.945

Aufgabe 2 - Clustering

Warum agglomeratives Clustering die beste Wahl ist

Ausschluss der Alternativen:

1. **Wir haben eine vorberechnete, nicht-euklidische Distanzmatrix.** Die kombinierte Distanz aus Stats (Euklidisch), Types (Jaccard) und Size (log-Euklidisch) ist nicht in einen euklidischen Vektorraum eingebettet. Verfahren, die intern Mittelwerte berechnen, sind damit nicht direkt anwendbar.
2. **k-Means scheidet aus.** k-Means setzt einen euklidischen Feature-Raum voraus.
3. **Ward-Linkage scheidet aus dem gleichen Grund aus.**
4. **Spektrales Clustering wäre möglich, ist aber schwerer zu tunen** für unseren Anwendungsfall überwiegt der Aufwand den Nutzen.

Auswahl 1: **Average Linkage funktioniert** ist der Kompromiss zwischen Single Linkage (anfällig für Ketten) und Complete Linkage (anfällig gegen Ausreißer) und liefert kompakte, etwa gleich große Cluster - genau das, was wir bei einem heterogenen Datensatz wollen.

Auswahl 2: **DBSCAN/HDBSCAN funktioniert** und wird als Vergleich herangezogen.

Wir durchsuchen `k` von 3 bis 500 und wählen das `k` mit dem Wert, bei dem der Silhouette Score nicht mehr stark steigt (letzte Steigung vor negativen Werten). Vorherige Version (verworfen durch Einsatz von obiger Lösung): Letzte große Steigung ($\geq 30\%$); `best_k = 15`

```
In [64]: from sklearn.cluster import AgglomerativeClustering
from sklearn.metrics import silhouette_score

scores = {}
labels_dict = {}

# 1. Alle Scores und Labels ganz normal berechnen
for k in range(3, 500):
    labels_k = AgglomerativeClustering(
        n_clusters=k, metric="precomputed", linkage="average"
    ).fit_predict(D)
```

```
sil = silhouette_score(D, labels_k, metric="precomputed")
scores[k] = sil
labels_dict[k] = labels_k

# 2. Den Peak finden (Letzter Anstieg vor der ersten negativen Steigung)
best_k = 3 # Wir starten standardmäßig bei 3

for k in range(4, 500):
    sprung = scores[k] - scores[k-1]

    if sprung > 0:
        # Der Score wird besser -> wir merken uns dieses k
        best_k = k
    elif sprung < 0:
        # Der Score wird schlechter (erste negative Steigung)
        # best_k bleibt der letzte Wert, bei dem es noch nach oben ging.
        break

# 3. Das beste Ergebnis zuweisen
best_sil = scores[best_k]
labels = labels_dict[best_k]

# 4. Übersichtliche Ausgabe
print(f"Bestes k (Peak vor dem ersten Abfall) = {best_k} (Silhouette = {best_sil:.4f})\n")

for k, s in list(scores.items())[:40]:
    if k > 3:
        jump = s - scores[k-1]
        jump_str = f" | Sprung: {jump:+8.4f}"
    else:
        jump_str = " " * 19

    marker = " <-- Bestes k (letzte positive Steigung)" if k == best_k else ""
    print(f"  k={k:>2}: Silhouette={s:.4f}{jump_str}{marker}")
```

Bestes k (Peak vor dem ersten Abfall) = 19 (Silhouette = 0.3592)

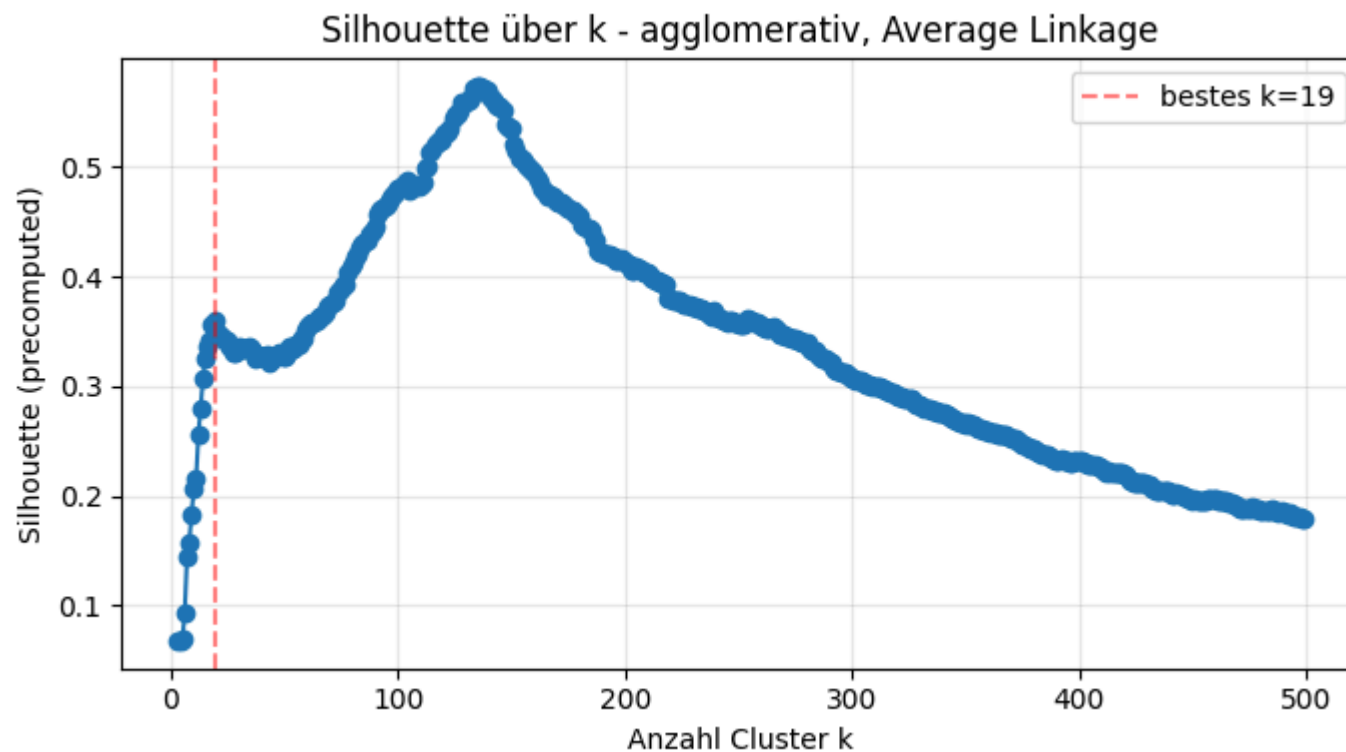
```

k= 3: Silhouette=0.0677
k= 4: Silhouette=0.0682 | Sprung: +0.0005
k= 5: Silhouette=0.0689 | Sprung: +0.0007
k= 6: Silhouette=0.0935 | Sprung: +0.0246
k= 7: Silhouette=0.1442 | Sprung: +0.0507
k= 8: Silhouette=0.1576 | Sprung: +0.0134
k= 9: Silhouette=0.1833 | Sprung: +0.0257
k=10: Silhouette=0.2065 | Sprung: +0.0232
k=11: Silhouette=0.2147 | Sprung: +0.0082
k=12: Silhouette=0.2551 | Sprung: +0.0404
k=13: Silhouette=0.2802 | Sprung: +0.0250
k=14: Silhouette=0.3062 | Sprung: +0.0260
k=15: Silhouette=0.3258 | Sprung: +0.0197
k=16: Silhouette=0.3369 | Sprung: +0.0111
k=17: Silhouette=0.3418 | Sprung: +0.0049
k=18: Silhouette=0.3560 | Sprung: +0.0142
k=19: Silhouette=0.3592 | Sprung: +0.0032 <-- Bestes k (letzte positive Steigung)
k=20: Silhouette=0.3498 | Sprung: -0.0094
k=21: Silhouette=0.3467 | Sprung: -0.0031
k=22: Silhouette=0.3446 | Sprung: -0.0021
k=23: Silhouette=0.3437 | Sprung: -0.0009
k=24: Silhouette=0.3413 | Sprung: -0.0024
k=25: Silhouette=0.3412 | Sprung: -0.0001
k=26: Silhouette=0.3359 | Sprung: -0.0053
k=27: Silhouette=0.3311 | Sprung: -0.0047
k=28: Silhouette=0.3307 | Sprung: -0.0004
k=29: Silhouette=0.3348 | Sprung: +0.0040
k=30: Silhouette=0.3362 | Sprung: +0.0015
k=31: Silhouette=0.3339 | Sprung: -0.0023
k=32: Silhouette=0.3348 | Sprung: +0.0009
k=33: Silhouette=0.3341 | Sprung: -0.0007
k=34: Silhouette=0.3369 | Sprung: +0.0028
k=35: Silhouette=0.3351 | Sprung: -0.0018
k=36: Silhouette=0.3302 | Sprung: -0.0049
k=37: Silhouette=0.3252 | Sprung: -0.0051
k=38: Silhouette=0.3291 | Sprung: +0.0039
k=39: Silhouette=0.3272 | Sprung: -0.0019
k=40: Silhouette=0.3265 | Sprung: -0.0006

```

k=41: Silhouette=0.3270 | Sprung: +0.0005
 k=42: Silhouette=0.3284 | Sprung: +0.0014

```
In [65]: fig, ax = plt.subplots(figsize=(7, 4))
ax.plot(list(scores.keys()), list(scores.values()), marker="o")
ax.axvline(best_k, color="red", linestyle="--", alpha=0.5, label=f"bestes k={best_k}")
ax.set_xlabel("Anzahl Cluster k")
ax.set_ylabel("Silhouette (precomputed)")
ax.set_title("Silhouette über k - agglomerativ, Average Linkage")
ax.legend()
ax.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```



HDBSCAN als Vergleich

HDBSCAN benötigt kein `k` - wir geben nur eine minimale Clustergröße vor. Punkte, die nicht passen, bekommen das Label `-1` (Rauschen), das wir später für die Ausreißererkennung wiederverwenden.

```
In [66]: try:
import hdbscan
hdb_labels = hdbscan.HDBSCAN(
    metric="precomputed", min_cluster_size=15, min_samples=5
).fit_predict(D.astype(np.float64))
n_clusters = len(set(hdb_labels)) - (1 if -1 in hdb_labels else 0)
n_noise = int((hdb_labels == -1).sum())
print(f"HDBSCAN: {n_clusters} Cluster, {n_noise} Rauschpunkte")
except ImportError:
hdb_labels = None
print("hdbscan nicht installiert - überspringe (pip install hdbscan)")
```

HDBSCAN: 14 Cluster, 367 Rauschpunkte

Aufgabe 3 - 2D-Visualisierung mit tSNE und UMAP

Beide Verfahren akzeptieren eine vorberechnete Distanzmatrix, deshalb können wir `D` direkt übergeben. Ziel der Plots: **visuell prüfen, ob die Cluster-Labels echter Struktur entsprechen.**

Parameter-Hinweise:

- **tSNE** `perplexity=30` ist ein sinnvoller Default für ~900 Punkte; niedrigere Werte zerfasern, höhere verschmieren.
- **UMAP** `n_neighbors=20`, `min_dist=0.1` erzeugt kompaktere, deutlicher abgesetzte Cluster als die Defaults.

```
In [67]: tsne_emb = TSNE(
    n_components=2, metric="precomputed", init="random",
    perplexity=30, random_state=42,
).fit_transform(D)

try:
import umap
umap_emb = umap.UMAP(
    n_components=2, metric="precomputed",
    n_neighbors=20, min_dist=0.1, random_state=42,
```

```

    ).fit_transform(D)
except ImportError:
    umap_emb = None
    print("umap-learn nicht installiert - überspringe UMAP (pip install umap-learn)")

```

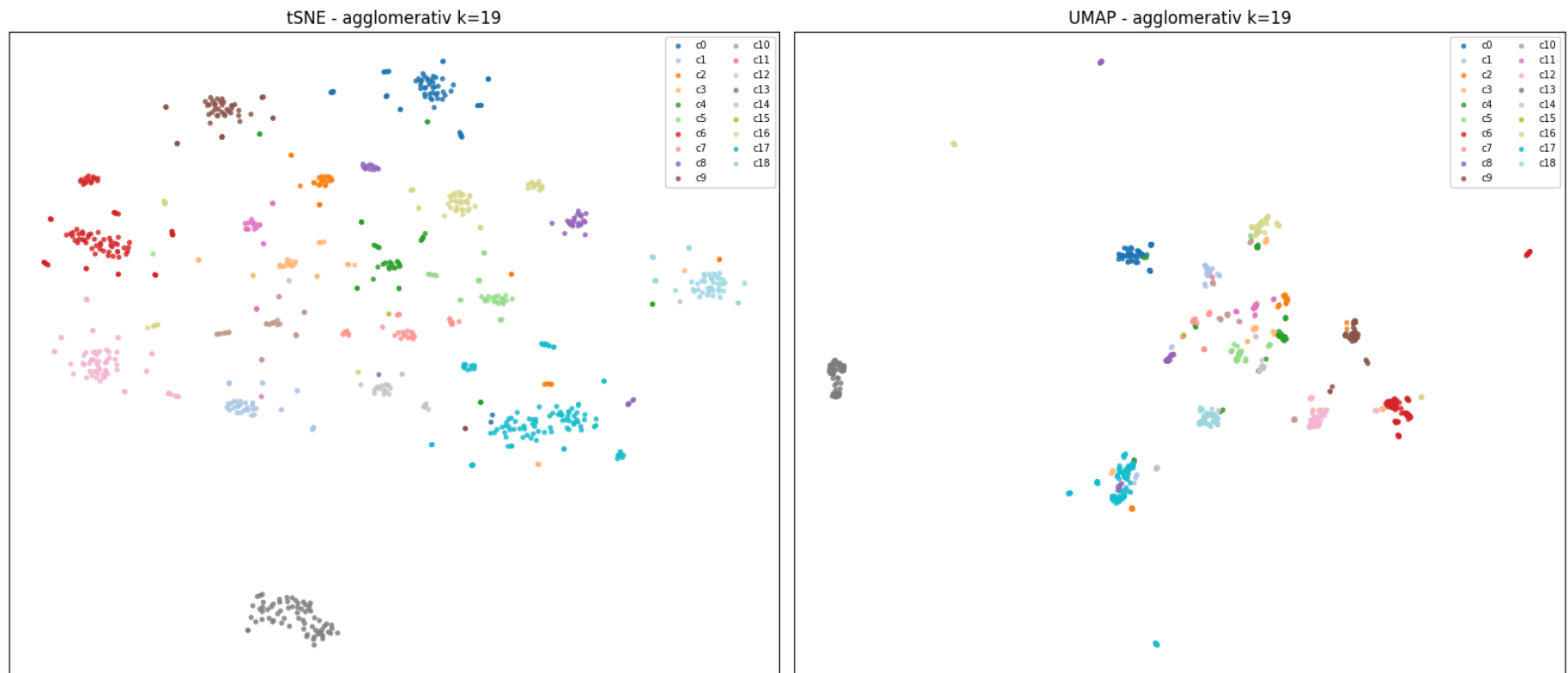
Agglomerative Clustering

```

In [68]: def scatter(ax, emb, labels, title, names=None, outliers=None):
    labels = np.asarray(labels)
    uniq = sorted(set(labels))
    cmap = plt.get_cmap("tab20", max(len(uniq), 1))
    for i, lab in enumerate(uniq):
        mask = labels == lab
        color = "lightgrey" if lab == -1 else cmap(i)
        name = "Rauschen" if lab == -1 else f"c{lab}"
        ax.scatter(emb[mask, 0], emb[mask, 1], s=14, c=[color],
                    label=name, alpha=0.85, edgecolors="none")
    if outliers is not None:
        ax.scatter(emb[outliers, 0], emb[outliers, 1], s=70,
                    facecolors="none", edgecolors="red", linewidths=1.5,
                    label="Ausreißer")
        if names is not None:
            for i in outliers:
                ax.annotate(names[i], (emb[i, 0], emb[i, 1]),
                            fontsize=7, alpha=0.7)
    ax.set_title(title)
    ax.set_xticks([]); ax.set_yticks([])
    ax.legend(loc="best", fontsize=7, ncol=2)

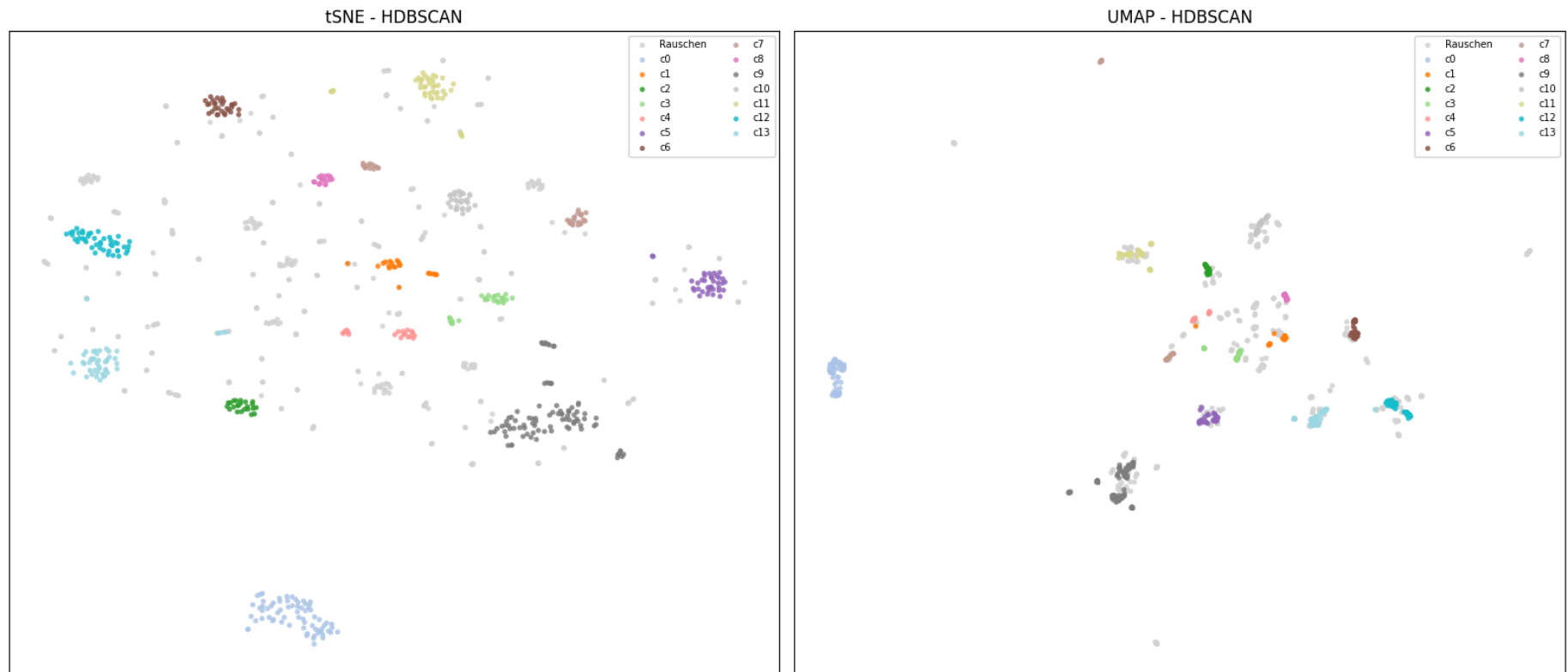
n_panels = 2 if umap_emb is not None else 1
fig, axes = plt.subplots(1, n_panels, figsize=(8 * n_panels, 7))
if n_panels == 1:
    axes = [axes]
scatter(axes[0], tsne_emb, labels, f"tSNE - agglomerativ k={best_k}")
if umap_emb is not None:
    scatter(axes[1], umap_emb, labels, f"UMAP - agglomerativ k={best_k}")
plt.tight_layout()
plt.savefig(OUT / "embeddings_agglom.png", dpi=130)
plt.show()

```



HDBSCAN

```
In [69]: if hdb_labels is not None:
    fig, axes = plt.subplots(1, n_panels, figsize=(8 * n_panels, 7))
    if n_panels == 1:
        axes = [axes]
    scatter(axes[0], tsne_emb, hdb_labels, "tSNE - HDBSCAN")
    if umap_emb is not None:
        scatter(axes[1], umap_emb, hdb_labels, "UMAP - HDBSCAN")
    plt.tight_layout()
    plt.savefig(OUT / "embeddings_hdbscan.png", dpi=130)
    plt.show()
```



Aufgabe 4 - Ausreißererkennung

Wir verwenden drei Verfahren und suchen nach **Unterschieden**:

- **Local Outlier Factor (LOF)** sucht den "Einsiedler am Rande des Dorfes". Gut darin, Pokemon zu finden, die in dünnen Regionen der Metrik liegen.
- **Isolation Forest** sucht denjenigen, die "auf einem komplett anderen Kontinent" leben.
- **HDBSCAN-Rauschen** (aus Aufgabe 2) - Punkte, die überhaupt keinem Cluster zugeordnet werden konnten.

Ein Pokemon, das von zwei oder drei Methoden markiert wird, ist ein starker Ausreißerkandidat.

```
In [70]: contamination = 0.03 # ~3 % erwartete Ausreißer
```

```

lof_mask = LocalOutlierFactor(
    metric="precomputed", n_neighbors=20, contamination=contamination
).fit_predict(D) == -1

iso_mask = IsolationForest(
    contamination=contamination, random_state=42
).fit_predict(D) == -1

print(f"LOF markiert          : {lof_mask.sum()}")
print(f"Isolation Forest markiert: {iso_mask.sum()}")
print(f"LOF UND ISO einiger : {(lof_mask & iso_mask).sum()}")
if hdb_labels is not None:
    print(f"HDBSCAN-Rauschen      : {(hdb_labels == -1).sum()}")

```

```

LOF markiert          : 27
Isolation Forest markiert: 27
LOF UND ISO einiger   : 3
HDBSCAN-Rauschen      : 367

```

```

In [71]: outlier_idx = np.where(lof_mask | iso_mask)[0]
rows = []
for i in outlier_idx:
    flags = []
    if lof_mask[i]: flags.append("LOF")
    if iso_mask[i]: flags.append("ISO")
    if hdb_labels is not None and hdb_labels[i] == -1: flags.append("HDB")

    rows.append({
        "name": df.loc[i, "name"],
        "stat_summe": int(df.loc[i, STAT_COLS].sum()),
        "groesse_m": df.loc[i, "height_m"],
        "gewicht_kg": df.loc[i, "weight_kg"],
        "typen": ", ".join(t.replace("type_", "") for t in TYPE_COLS if df.loc[i, t] == 1),
        "flags": "+".join(flags),
        "n_flags": len(flags),
    })

outliers_df = pd.DataFrame(rows).sort_values(["n_flags", "name"], ascending=[False, True])

# Nur die Ausreißer anzeigen, die von mindestens 2 Algorithmen gefunden wurden
outliers_df[outliers_df["n_flags"] >= 2]

```

Out[71]:

	name	stat_summe	groesse_m	gewicht_kg	typen	flags	n_flags
44	Kartana	570	0.3	0.1	grass,steel	LOF+ISO+HDB	3
41	Araquanid	454	1.8	82.0	water,bug	LOF+HDB	2
37	Avalugg	514	2.0	505.0	ice	ISO+HDB	2
11	Blissey	540	1.5	46.8		LOF+ISO	2
43	Bruxish	475	0.9	19.0	water,psychic	LOF+HDB	2
18	Carvanha	305	0.8	20.8	water,dark	LOF+HDB	2
4	Chansey	450	1.1	34.6		LOF+ISO	2
5	Chinchou	330	0.5	12.0	water,electric	LOF+HDB	2
20	Crawdaunt	468	1.1	32.8	water,dark	LOF+HDB	2
28	Empoleon	530	1.7	84.5	water,steel	LOF+HDB	2
48	Eternatus	690	20.0	950.0	poison,dragon	ISO+HDB	2
34	Frillish	335	1.2	33.0	water,ghost	LOF+HDB	2
32	Giratina	680	4.5	750.0	ghost,dragon	ISO+HDB	2
45	Guzzlord	570	5.5	888.0	dragon,dark	ISO+HDB	2
35	Jellicent	480	2.2	135.0	water,ghost	LOF+HDB	2
36	Keldeo	580	1.4	48.5	water,fighting	LOF+HDB	2
6	Lanturn	460	1.2	22.5	water,electric	LOF+HDB	2
21	Lileep	355	1.0	23.8	grass,rock	LOF+HDB	2
14	Lombre	340	1.2	32.5	water,grass	LOF+HDB	2
13	Lotad	220	0.5	2.6	water,grass	LOF+HDB	2
15	Ludicolo	480	1.5	55.0	water,grass	LOF+HDB	2
12	Lugia	680	5.2	216.0	flying,psychic	ISO+HDB	2

	name	stat_summe	groesse_m	gewicht_kg	typen	flags	n_flags
7	Marill	250	0.4	8.5	water,fairy	LOF+HDB	2
47	Melmetal	600	2.5	800.0	steel	ISO+HDB	2
0	Poliwrath	510	1.3	54.0	water,fighting	LOF+HDB	2
27	Rayquaza	680	7.0	206.5	flying,dragon	ISO+HDB	2
25	Regice	580	1.8	175.0	ice	ISO+HDB	2
42	Salazzle	480	1.2	22.2	fire,poison	LOF+HDB	2
10	Shuckle	505	0.6	20.5	bug,rock	ISO+HDB	2
2	Slowpoke	315	1.2	36.0	water,psychic	LOF+HDB	2
23	Spheal	290	0.8	39.5	water,ice	LOF+HDB	2
9	Steelix	510	9.2	400.0	ground,steel	ISO+HDB	2
16	Surskit	269	0.5	1.7	water,bug	LOF+HDB	2
38	Volcanion	600	1.7	195.0	fire,water	LOF+HDB	2

Ausreißer im 2D-Embedding

Wir heben die markierten Pokemon in den tSNE-/UMAP-Plots hervor.

```
In [72]: names = df["name"].tolist()

fig, axes = plt.subplots(n_panels, 1, figsize=(14, 10 * n_panels))

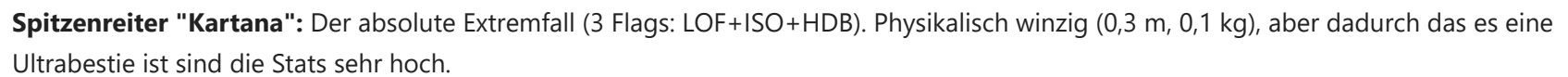
if n_panels == 1:
    axes = [axes]

scatter(axes[0], tsne_emb, labels, "tSNE - Ausreißer markiert",
        names=names, outliers=outlier_idx)

if umap_emb is not None and n_panels > 1:
```

```
scatter(axes[1], umap_emb, labels, "UMAP - Ausreißer markiert",  
        names=names, outliers=outlier_idx)  
  
plt.tight_layout()  
plt.savefig(OUT / "embeddings_outliers.png", dpi=130)  
plt.show()
```



Die 4 Ausreißer-Muster:

- **1. Extreme Dimensionen (Größe & Gewicht)**
 - *Giganten*: Eternatus (20,0 m, 950,0 kg), Steelix (9,2 m, 400,0 kg), Guzzlord (888,0 kg), Melmetal (800,0 kg)
 - *Winzlinge*: Kartana (siehe oben), Surskit (1,7 kg, 0,5 m und Stat-Summe 269)
- **2. Einzigartige Typ-Kombinationen**
 - *Widersprüchlich*: Volcanion (Feuer/Wasser, zusätzlich Stat-Summe: 600 – liegt elementar völlig isoliert)
 - *Exotisch*: Salazzle (Feuer/Gift), Empoleon (Wasser/Stahl), Lileep (Pflanze/Gestein), Jellicent (Wasser/Geist)
- **3. Extreme Stat-Profile (Min-Maxing)**
 - *KP-Monster*: Chansey (Stat-Summe 450) & Blissey (Stat-Summe 540) – Astronomische KP, aber Werte nahe Null in der physischen Verteidigung.
 - *Verteidigungs-Monster*: Shuckle (Stat-Summe 505 – fast ausschließlich Defensive bei minimaler Offensive), Avalugg (Stat-Summe 514, 505,0 kg Gewicht).
- **4. Legendäre Pokemon**
 - *Legendaries*: Rayquaza (Stat-Summe 680, 7,0 m), Lugia (Stat-Summe 680, 5,2 m), Giratina (Stat-Summe 680, 750,0 kg).

Fazit: Die Algorithmen erkennen zielsicher "Regelbrecher" im Game-Design

Aufgabe 5 - Diskussion und Reflexion

Was lief gut?

- **t-SNE Visualisierung:** Sehr einfache Umsetzung. Mit minimalem Parameter-Tuning (z. B. `perplexity=30`) lieferte t-SNE sofort sehr gute, sichtbare Strukturen – deutlich besser als UMAP.
- **"Back and forth"-Workflow:** Das stetige Testen, Anpassen und Vergleichen verschiedener Distanzfunktionen und Algorithmen hat die Modellqualität iterativ stark verbessert.
- **Robuste Ausreißer-Erkennung:** Die Kombination mehrerer Methoden zu einer Konsensliste lieferte extrem präzise und intuitiv treffsichere Resultate.

Herausforderungen

- **Design der Distanzfunktion (subjektiv):**
 - *Gewichtung:* Aufteilung (Stats, Typen, Größe) ist plausibel, aber subjektiv.
 - *Heavy Tails:* Extreme Ausreißer in Größe/Gewicht erforderten Log-Skalierung (`log1p`), sonst dominieren sie die Metrik.
- **Optimales `k` finden (Silhouette Score):**
 - *Problem:* Score steigt oft monoton, kein klares globales Maximum.
 - *Alter Ansatz (verworfen):* Starre Suche nach letztem großen Sprung ($\geq 30\%$) liefert guten Wert ($k=15$), aber wurde durch neue Logik verworfen.
 - *Neue Logik:* Systematische Iteration ($k=3$ bis 500). **Bestes `k` = letzter Peak vor dem ersten Abfall** (letzte positive Steigung). Erkennt dynamisch und robust den besten Cut-off.

Mit mehr Zeit

- **Thumbnails verwenden** - die Bilder der Pokemon verwenden und interaktive Visualisierung
- **Distanzfunktion(-Gewichte) systematisch tunen**
- **Mehr Clusterverfahren vergleichen**
- **Analyse pro Generation** - verschiedene Pokemon-Generationen vergleichen (können über ID der Generation zugeordnet werden)